SOLID PRINCIPLES

COMPLETE HANDBOOK

EVERY DEVELOPER SHOULD KNOW

★ What are SOLID Principles ?

SOLID is a set of 5 design principles introduced by Robert C. Martin (Uncle Bob) to help developers write clean, maintainable, scalable and extensible code.

- S. - Single Responsibility Principle
- O. - Open Closed Principle
- L. - Liskov Substitution Principle
- I. - Interface Segregation Principle
- D. - Dependency Inversion Principle

★ Why was SOLID introduced ?

- To solve problems in complex software systems.
- To reduce the impact of changes.
- To make code easy to understand.
- To improve code reusability and flexibility.
- To follow good object-oriented design.

Benefits of SOLID Principles

- Code becomes easy to maintain.
- Changes can be made without breaking existing functionality.
- Improved code reusability.
- Easier testing and debugging.
- Better scalability and flexibility.

Impact of Using SOLID Principles

Bad Design & Tight Coupling → Hard to Maintain, Hard to Test, Hard to Extend → Apply SOLID Principles → Clean Code, Flexible, Easy to Maintain

Simple Example (Before applying SOLID)

```
class UserService {
    void registerUser(User user) {
        // Save user to database
        saveUserToDatabase(user);
        // Send welcome email
        sendEmail(user);
        // Generate report
        generateReport(user);
    }
}
```

→ In this example, UserService class is doing multiple jobs.

- If database logic changes, we modify this class.
- If email logic changes, we modify this class.
- If report logic changes, we modify this class.

SOLID is not about writing more code, it's about writing the right code.

In the next pages, we will see each principle in detail with diagrams, real examples and Java code.

S.O.L.I.D PRINCIPLES

COMPLETE HANDBOOK

EVERY DEVELOPER SHOULD KNOW

S. - Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have only one job or responsibility.

Problem (Violation of SRP)

```
class InvoiceService {
    void saveInvoice(Invoice inv) {
        // save invoice to DB
    }
    void sendEmail(Invoice inv) {
        // send invoice email
    }
    void printInvoice(Invoice inv) {
        // print invoice to file
}
```

- This class has 3 different responsibilities.
- Any change in email or printing logic will affect the class.
- Hard to maintain, test and extend.

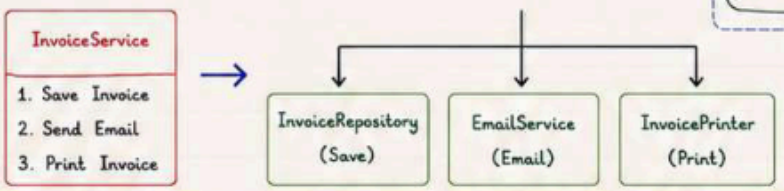
Solution (Follow SRP)

```
class InvoiceRepository {
    void save(Invoice inv) {
        // save invoice to DB
    }
}
```
// Responsible only for saving invoices

```
class EmailService {
    void sendEmail(Invoice inv) {
        // send invoice email
    }
}
```
// Responsible only for sending emails

```
class InvoicePrinter {
    void print(Invoice inv) {
        // print invoice to file
    }
}
```
// Responsible only for printing invoices

Visual Representation



Real World Example

In an e-commerce application:

- OrderService should only handle order logic.
- EmailService should only handle notifications.
- PaymentService should only handle payments.

This separation makes the system easier to manage and scale.

Benefits

- Code is easier to understand.
- Changes in one responsibility don't affect others.
- Easier to test (unit testing).
- Better maintainability and flexibility.
- Promotes clean and modular code.

Key Takeaway : Don't put multiple jobs in a single class.

One class - One responsibility - One reason to change.

S.O.L.I.D PRINCIPLES

COMPLETE HANDBOOK

EVERY DEVELOPER SHOULD KNOW

O. - Open Closed Principle (OCP)

Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

Problem (Violation of OCP)

```
class PaymentService {
    void pay(String type, double amount) {
        if(type.equals("UPI")) {
            // process UPI payment
        } else if(type.equals("CARD")) {
            // process Card payment
        } else if(type.equals("WALLET")) {
            // process Wallet payment
        }
    }
}
```

- Every time a new payment method is added, we need to modify this class.
- Class is not closed for modification.

Solution (Follow OCP)

1. Use abstraction (interface)

```
interface Payment {
    void pay(double amount);
}
```

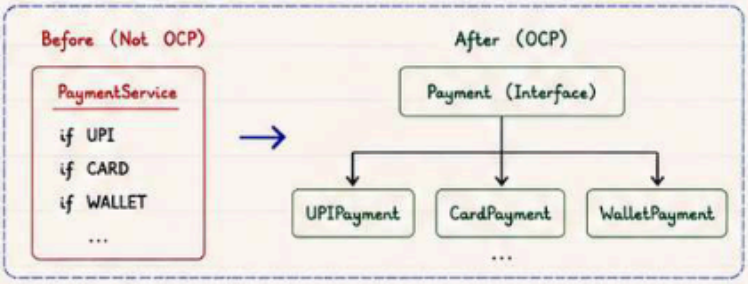
// Open for extension

2. Implement for each payment type

```
class UPIPayment implements Payment {
    public void pay(double amount) {
        // process UPI payment
    }
}
class CardPayment implements Payment {
    public void pay(double amount) {
        // process Card payment
    }
}
class WalletPayment implements Payment {
    public void pay(double amount) {
        // process Wallet payment
    }
}
```

// Extend by adding new classes, not by changing existing code.

Visual Representation



Real World Example

In an e-commerce application:
Today we support UPI and Card payments.
Tomorrow we may add PayPal, NetBanking, Crypto etc.
If we follow OCP, we just create new classes for new payment methods without changing existing code.

Benefits

- Easy to add new functionality.
- Minimizes the risk of modifying existing code.
- Follows the open for extension and closed for modification philosophy.
- Leads to more stable and maintainable systems.

Key Takeaway : Design your code in such a way that new features can be added by extending the code, not by changing the existing code.

S.O.L.I.D PRINCIPLES

COMPLETE HANDBOOK

EVERY DEVELOPER SHOULD KNOW

L. - Liskov Substitution Principle (LSP)

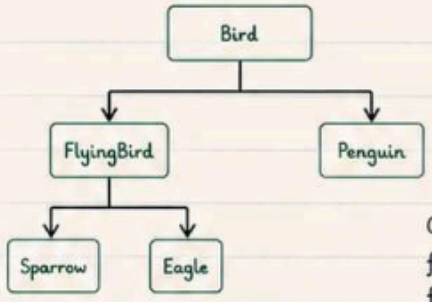
Objects of a subclass should be replaceable with objects of its superclass without breaking the application.

Problem (Violation of LSP)

```
class Bird {
    void fly() {
        System.out.println("I can fly");
    }
}
class Sparrow extends Bird { }
class Penguin extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperation
            Exception("I can't fly");
    }
}
```

- Penguin is a Bird but it can't fly.
- Replacing Bird object with Penguin breaks the program.
- This violates LSP.

Solution (Follow LSP)

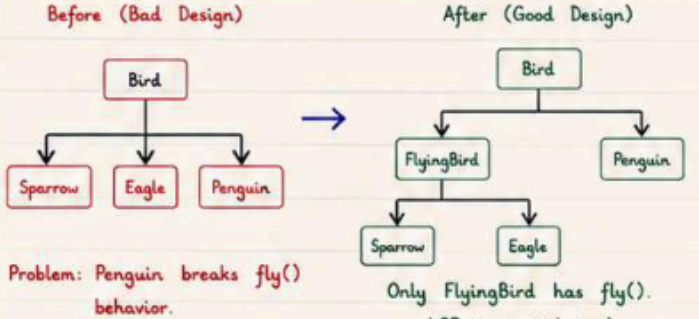


Only birds that can fly should have fly() behavior.

```
class Bird { }   // General
class FlyingBird extends Bird {
    void fly() {
        System.out.println("I can fly");
    }
}
class Sparrow extends FlyingBird { }
class Eagle extends FlyingBird { }
class Penguin extends Bird {
    void swim() {
        System.out.println("I can swim");
    }
}
```

- Now any Bird object can replace another Bird object without breaking the application.
- LSP is followed.

Visual Representation



Real World Example

In a payment system, if we have a base class Payment and subclasses CreditCardPayment, UpiPayment, NetBankingPayment, each subclass should behave like a Payment without causing errors in the system.
If any subclass changes the expected behavior, it violates LSP.

Benefits

- Makes the system more reliable.
- Reduces bugs caused by incorrect inheritance.
- Improves maintainability and extensibility.
- Encourages proper use of inheritance.

Key Takeaway : Subclasses should extend the behavior of parent classes, not change or break it.

S.O.L.I.D PRINCIPLES

COMPLETE HANDBOOK

EVERY DEVELOPER SHOULD KNOW

I. - Interface Segregation Principle (ISP)

No client should be forced to depend on methods it does not use.

Create small, specific interfaces instead of one large generic interface.

Problem (Violation of ISP)

```
interface Worker {
    void work();
    void eat();
    void sleep();
}

class Robot implements Worker {
    public void work() {
        // Robot does work
    }
    public void eat() {
        // Robot doesn't eat
        throw new UnsupportedOperationException();
    }
    public void sleep() {
        // Robot doesn't sleep
        throw new UnsupportedOperationException();
    }
}
```

- Robot class is forced to implement methods it doesn't need.
- Unnecessary methods lead to useless code and maintenance problems.
- This violates ISP.

Solution (Follow ISP)

Step 1: Create specific interfaces

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

interface Sleepable {
    void sleep();
}
```

Step 2: Implement only required interfaces

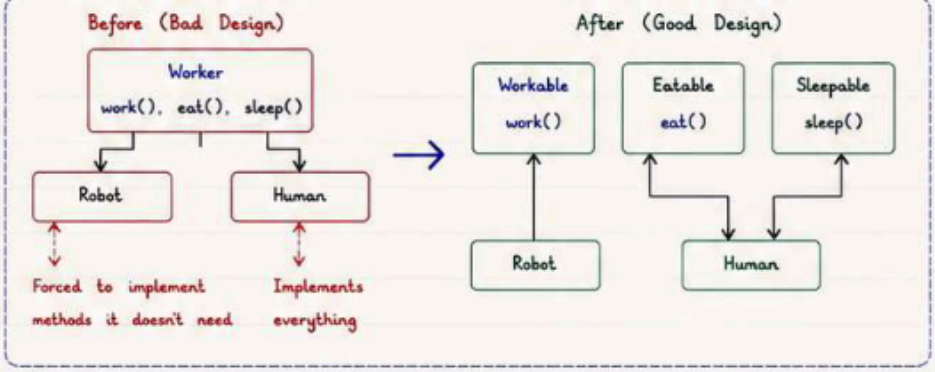
```
class Robot implements Workable {
    public void work() {
        // Robot does work
    }
}
```

```
class Human implements Workable, Eatable, Sleepable {
    public void work() {
        // Human works
    }
    public void eat() {
        // Human eats
    }
    public void sleep() {
        // Human sleeps
    }
}
```

Step 3: Client depends only on the interfaces it needs

- Robot → Workable
- Human → Workable, Eatable, Sleepable

Visual Representation



Real World Example

In a restaurant system:

- Chef → cooks food (work)
- Waiter → serves food (work)
- Manager → manages & reports (work)
- Not everyone needs to do everything.

Using ISP, we create small interfaces:
Cookable, Servable, Reportable, etc.
Each role implements only what it actually needs.

Benefits

- Clients are not forced to depend on unused methods.
- Interfaces remain small, focused and easy to understand.
- Code becomes cleaner and easier to maintain.
- Reduces the impact of changes in other parts of the system.
- Improves flexibility and scalability.

Example in Java Collections

Java follows ISP:

- Iterable → has iterator()
- Collection → has size(), add(), remove(), etc.
- List → has get(int), set(int), etc.

Each interface is small and specific.

Key Takeaway : Prefer many small, client-specific interfaces over one large, generic interface.

S.O.L.I.D PRINCIPLES

D. - Dependency Inversion Principle (DIP)

High level modules should not depend on low level modules.
Both should depend on abstractions (interfaces).

Problem (Violation of DIP)

```
class OrderService {
    private MySQLDatabase db = new MySQLDatabase();

    public void saveOrder(Order order) {
        db.save(order);
    }
}

class MySQLDatabase {
    public void save(Order order) {
        // save to MySQL
    }
}
```

- OrderService depends on concrete class MySQLDatabase.
- If we want to use MongoDB or any other DB, we must modify OrderService.
- Hard to test and not flexible.
- This violates DIP.

Solution (Follow DIP)

```
// 1. Create Abstraction (Interface)
interface OrderRepository {
    void save(Order order);
}
```

High level module depends on abstraction.

```
// 2. Implement Abstraction
class MySQLOrderRepository implements OrderRepository {
    public void save(Order order) {
        // save to MySQL
    }
}

class MongoOrderRepository implements OrderRepository {
    public void save(Order order) {
        // save to MongoDB
    }
}
```

```
// 3. Depend on Abstraction
class OrderService {
    private OrderRepository repo;

    // Constructor Injection
    public OrderService(OrderRepository repo) {
        this.repo = repo;
    }

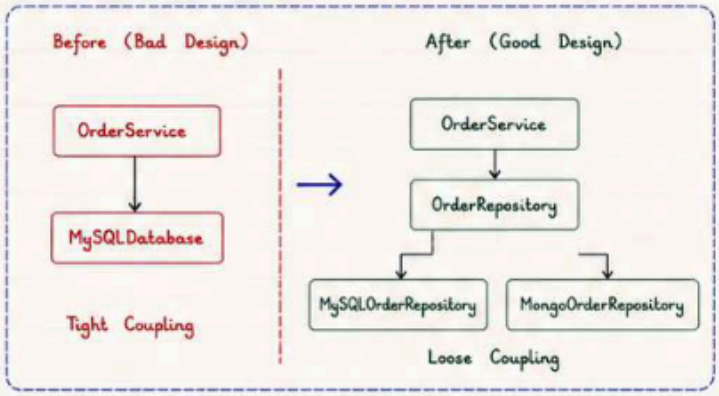
    public void saveOrder(Order order) {
        repo.save(order);
    }
}
```

Low level modules implement abstraction.

Easy to switch DB without changing OrderService.

Easy to test using mocks.

Visual Representation



Real World Example

- In a Notification system, we may send Email, SMS, Push etc.
- If we depend on concrete classes (EmailService, SMSService), changing the channel becomes difficult.
- Using DIP, we create an interface NotificationService.
- Different channels (EmailNotification, SMSNotification, PushNotification) implement this interface.
- Our business logic depends on NotificationService, not on any specific channel.

Benefits

- Reduces coupling between high level and low level modules.
- Code becomes flexible and easy to extend.
- Easy to switch implementations (e.g., MySQL → MongoDB).
- Improves testability (use mock implementations).
- Follows dependency injection principle (used in Spring).
- Makes the system more maintainable and scalable.

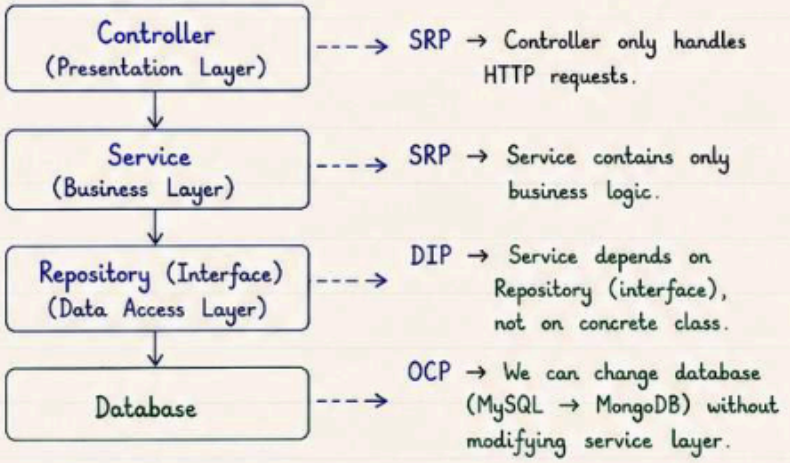
Key Takeaway : Depend on abstractions, not on concrete implementations.
This makes your code flexible, testable and easy to maintain.

S.O.L.I.D PRINCIPLES

SOLID Principles – At a Glance

Principle	In Short	What it Helps	Key Benefit	Think
S. - SRP	One class should have one reason to change.	Clean separation of responsibilities.	Easier to maintain and test.	One class One Job
O. - OCP	Open for extension, closed for modification.	Add new functionality without changing existing code.	Reduces bugs and improves stability.	Extend Don't Modify
L. - LSP	Subclasses should be replaceable with their parent classes.	Ensure correct inheritance and polymorphism.	Prevents unexpected behavior.	Is-A relationship must hold true
I. - ISP	Don't force clients to depend on methods they don't use.	Break large interfaces into smaller, focused ones.	Avoids unnecessary dependencies.	Many small interfaces are better
D. - DIP	Depend on abstractions, not on concrete implementations.	High level modules dependent on abstractions.	Improves flexibility and testability.	Program to Interface, not Implementation

Where Each Principle Applies in a Typical Spring Boot Project



SOLID Decision Guide

- Class doing too much? → S.R.P
- Need to add new feature frequently? → O.C.P
- Child class breaking parent behavior? → L.S.P
- Interface has unused methods? → I.S.P
- Using concrete class directly? → D.I.P

Final Thoughts

- SOLID is not a set of strict rules.
- It is a guideline to write clean, flexible and maintainable code.
- Start small, apply one principle at a time.
- Good design today → Strong system tomorrow.

"SOLID code is not about impressing others, it's about making your future self thank you."

Keep Remembering

S. – One Responsibility
O. – Extend, Don't Modify
L. – Replace Parent Safely
I. – Small Interfaces
D. – Depend on Abstractions

Follow SOLID and your code will be maintainable, testable and scalable.